EuroGlobe Bank — Implementation Feasibility Proposal

We will implement all Must Have and Should Have requirements of the EuroGlobe Bank specification, and both Could Have items in simplified form, in C#. We will use AI-assisted development. Only things that need real external systems are mocked: KYC/AML, the credit bureau, State Registers, insurers and dealers.

Must Have

Requirement	What we will deliver
Multi-track registration and KYC	Local / foreign / business registration forms. KYC/AML and State Registers will be mocked (auto-approve, with a configurable reject rule). No document upload processing, only file storage.
Multi-currency accounts and instant transfers	Will be fully implemented as in the documentation.
Smart mortgages	Mortgage application with automated scoring (DTI ≤ 35%, LTV ≤ 85%); the credit bureau check will be mocked. Fixed and variable rate selection, amortization schedule and early repayment (reduce term or reduce monthly payment). Full lifecycle: UnderReview → Approved → Active ↔ Delinquent → Closed. Euribor will be a configurable value instead of a live feed; changing it updates variable-rate mortgages. A daily background job will detect missed payments and move the mortgage to Delinquent, and back to Active once paid.
Car leasing	Vehicle catalog with pre-loaded cars, step-by-step contract configuration (duration, mileage, insurance, down payment) and monthly payment calculation. Approval checks balance, insurance and dealer availability; the insurance and dealer services will be mocked. Monthly billing will run automatically as a background job.
Multi-signature shared vaults	Will be fully implemented as in the documentation: vault creation with at least 3 members, member invitations, withdrawal requests with reserved funds, approve/reject,

	execution once the quorum is reached, and automatic expiry after 24 hours.
Automated micro-investments	Card payments are rounded up to the next whole euro and the difference is invested asynchronously after the payment completes. Asset prices (Bitcoin, S&P 500 ETF) will be fetched from public market data APIs, with the last known price used if the API is unavailable.
Card issuance and management	Will be fully implemented as in the documentation: virtual and physical card issuance (physical cards are not actually produced), daily spending limits, freeze/unfreeze and PIN change.

Should Have, Could Have and Won't Have

Requirement	What we will deliver
Investment analytics dashboard	Will be implemented: portfolio value, allocation per asset, historical performance chart and yield estimate based on market prices.
Manual capital investments	Will be implemented in a simplified form: users can manually buy fractional amounts of the same assets (Bitcoin, S&P 500 ETF) at current market prices.
Supplemental insurance packages	Will be implemented as optional add-ons (extended warranty, travel coverage) selectable during leasing configuration; no real insurer integration.
Won't Have items	Not implemented, as stated in the documentation.

Non-functional requirements

Requirement	What we will deliver
Encryption in transit	HTTPS for all client–server traffic.
Encryption at rest (AES-256, HSM)	Sensitive fields (personal codes, document numbers) encrypted with AES-256; passwords and PINs hashed. Keys stored in configuration instead of an HSM.

2FA / SCA	Implemented with TOTP (authenticator app), as shown in the wireframes, for login and high-risk actions (transfers, vault approvals, contract signing). Hardware keys and biometrics will not be supported.
Immutable audit log	Append-only log of state changes and money movements.
Fault isolation	Investment failures will not affect payments, since investments run asynchronously.
Performance, scalability, uptime, RPO/RTO	Not implemented; these targets cannot be measured in our project environment.